

Software Project - Report

EE25BTECH11043 - Nishid Khandagre

Summary of Strang's Video

Every matrix A can be expressed as:

$$A = U\Sigma V^T$$

where U and V are orthogonal matrices whose columns are orthonormal vectors, Σ is a diagonal matrix with entries σ_i called the **singular values**.

$$A^T A = V(\Sigma^T \Sigma)V^T$$

$$A A^T = U(\Sigma \Sigma^T)U^T$$

Properties

- $A v_i = \sigma_i u_i$
- The column vectors v_i are eigenvectors of $A^T A$:
- The column vectors u_i are eigenvectors of $A A^T$:

Algorithm Used: Jacobian

The Jacobian method is an iterative algorithm for computing the Singular Value Decomposition (SVD) of a matrix A .

It applies rotations to eliminate off-diagonal elements of $A^T A$, and converting it into a diagonal matrix. The diagonal elements of the obtained diagonal matrix represent the square of the singular values of A .

$$A = U\Sigma V^T$$

$$A^T A = V\Sigma^T \Sigma V^T$$

$$U = A V \Sigma^{-1}$$

Here, V and U are orthogonal matrices containing right and left singular vectors, and Σ is the diagonal matrix of singular values.

Explanation

1) Form the symmetric matrix

$$M = A^T A$$

2) Find the largest off-diagonal element

Identify indices (p, q) for the largest off-diagonal element where $p < q$.

Using this element we will find the angle.

3) Calculate the rotation angle

The rotation angle θ is chosen such that the new matrix has a zero at position (p, q)

$$\tan(2\theta) = \frac{2M_{pq}}{M_{qq} - M_{pp}}$$

Then

$$c = \cos(\theta), \quad s = \sin(\theta)$$

4) Form the orthogonal rotation matrix S

Construct an identity matrix and modify the four elements corresponding to indices (p, q) :

$$S_{pp} = S_{qq} = c, \quad S_{pq} = s, \quad S_{qp} = -s$$

All other diagonal entries are 1 and the rest are 0.

5) Find the transformation matrix

$$R = S^T M S$$

By This operation the off-diagonal term M_{pq} placed at (p, q) becomes zero in R .

6) Iterate

Repeat steps 1 to 4 for different pairs (p, q) until all off-diagonal elements become small or zero

7) Obtain eigenvalues

When the algorithm is done, R becomes approximately diagonal. where each diagonal entry $\lambda_i = R_{ii}$ is an eigenvalue of the original matrix.

8) Compute eigenvectors

The final orthogonal transformation matrix is obtained by successively multiplying all rotation matrices used:

$$V = S_1 S_2 S_3 \cdots S_k$$

Each time we multiply the rotation orthogonal matrix with V

The columns of V are the normalized eigenvectors corresponding to the eigenvalues λ_i .

Pseudocode of the Algorithm

1) Read Input Image:

Read the grayscale image and store it as matrix A and also normalize each entry of A .

Extract number of rows r , columns c , and maximum pixel value.

2) Compute Transpose:

Compute the transpose of the image matrix:

3) Form Symmetric Matrix:

Construct the symmetric matrix $A^T A$:

$$B = A^T A$$

4) Initialize Arrays that will stor the eigen vectors:

$$V = I$$

5) Perform Jacobian algorithm

Repeat until convergence or until the iteration limit is reached:

- a) Find indices of maximum off diagonal element (p, q)
- b) If max off-diagonal element is very small , stop iteration.
- c) Compute rotation angle:

$$\theta = \frac{1}{2} \tan^{-1} \left(\frac{2B_{pq}}{B_{qq} - B_{pp}} \right)$$

- d) Compute cosine and sine of the angle:

$$c = \cos(\theta), \quad s = \sin(\theta)$$

- e) Form the rotation orthogonal matrix S and then find the transformation matrix

$$B = S^T B S$$

- f) Update eigenvector matrix:

$$\mathbf{V} = \mathbf{V} S$$

After convergence, the diagonal elements of B contain eigenvalues:

- 6) **Sort Eigenvalues and Vectors:** Sort eigenvalues λ_i in descending order and reorder columns of $\mathbf{V}$ accordingly.
- 7) **Compute Singular Values:** The singular values are obtained as:

$$\sigma_i = \sqrt{\lambda_i}$$

forming the diagonal matrix:

$$\Sigma = \text{diag}(\sigma_1, \sigma_2, \dots, \sigma_c)$$

- 8) **Compute Left Singular Vectors:** Each left singular vector is computed as:

$$\mathbf{u}_j = \frac{A \mathbf{v}_j}{\sigma_j}$$

- 9) **Construct top k Matrix:** Using the top k singular values and vectors:

$$A_k = \sum_{t=1}^k \sigma_t \mathbf{u}_t \mathbf{v}_t^T$$

- 10) **Calculate Frobenius Error:**

$$F = \sqrt{\sum_{i=1}^r \sum_{j=1}^c (A_{ij} - A_{ij}^k)^2}$$

- 11) **Save Compressed Image:** Save the reconstructed image matrix

Comparison of Different Algorithms

- 1) **Householder Bidiagonalization and QR Algorithm**

The algorithm first reduces the input matrix to a bidiagonal form using Householder reflections, and then applies iterative QR transformations to extract singular values and vectors. It is highly efficient but it is complex to implement manually.

2) **Power Iteration**

These methods compute one eigenvalue and eigenvector at a time by iteratively applying the matrix to a random vector until convergence. power iteration is computationally slow and can miss smaller eigenvalues.

3) **Jacobian Method**

The Jacobi algorithm applies a series of orthogonal rotations to eliminate off-diagonal elements of a symmetric matrix. It is simple to understand and implement, guarantees numerical stability, and always produces an orthogonal eigenvector matrix.

Why Jacobi Method was Chosen

- it is easy to implement, and conceptually easy.
- It provides a clear understanding of how eigenvalues and eigenvectors are obtained through orthogonal transformations.
- Each step involves simple arithmetic and trigonometric operations, making it simple to understand.

Reconstructed images

For `einstein.pgm` - Original image

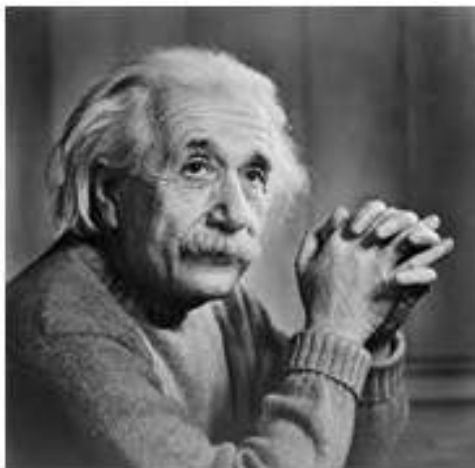


Fig. 3

1) For $k=20$

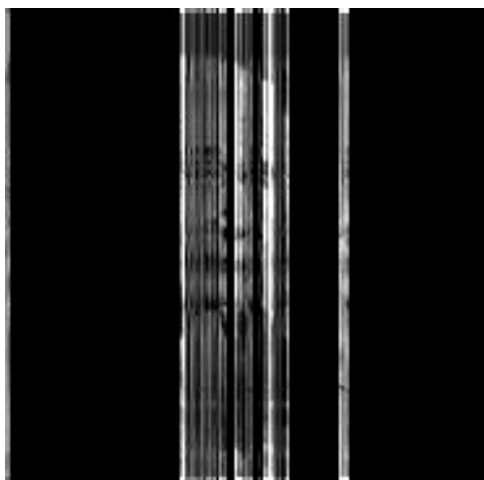


Fig. 1

2) For $k=50$



Fig. 2

3) For $k=100$



Fig. 3

4) For $k=180$



Fig. 4

For globe.pgm - Original image



Fig. 4

1) For $k=200$

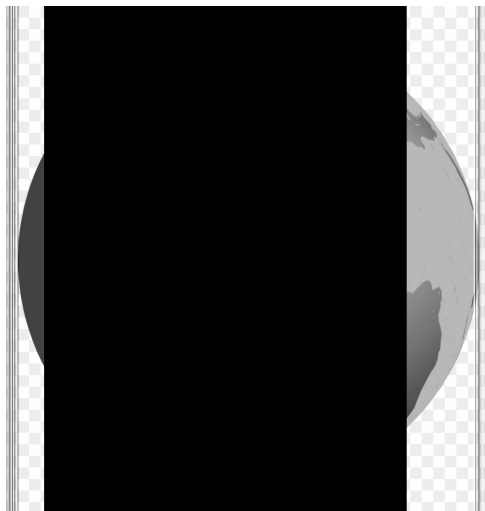


Fig. 1

2) For $k=400$

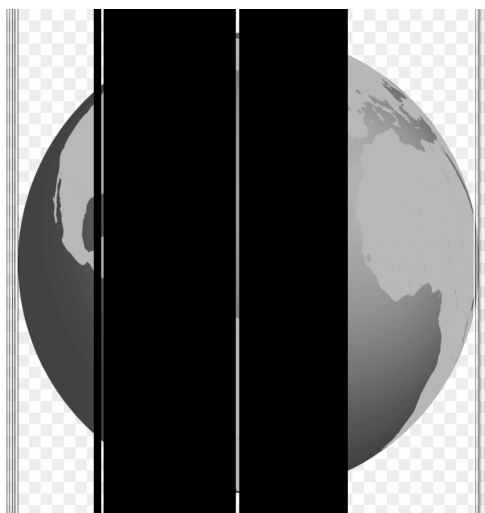


Fig. 2

3) For $k=600$



Fig. 3

4) For $k=800$

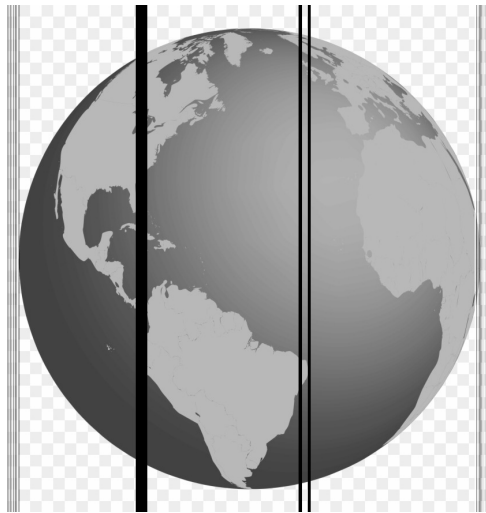


Fig. 4

For greyscale.pgm - Original image

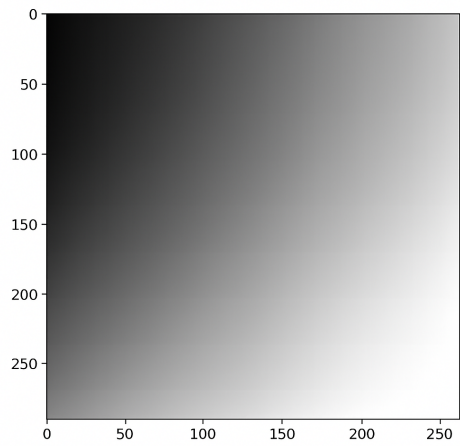


Fig. 4

1) For $k=400$

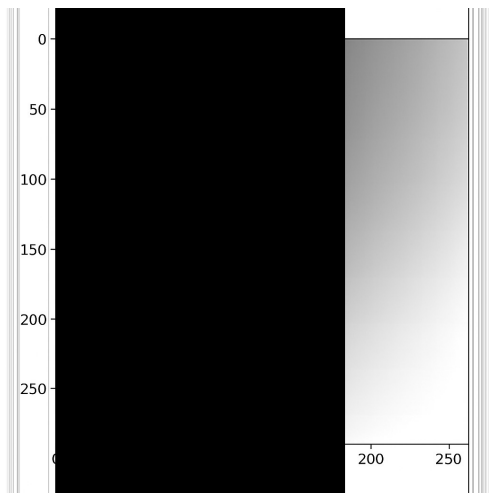


Fig. 1

2) For $k=600$

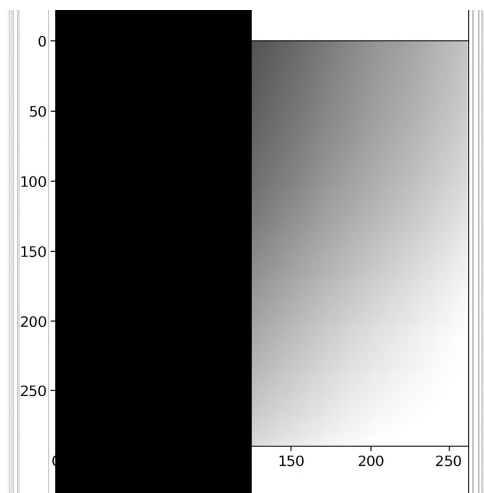


Fig. 2

3) For $k=800$

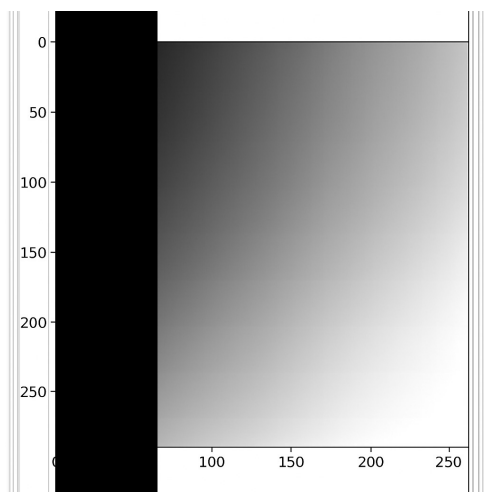


Fig. 3

4) For $k=1000$

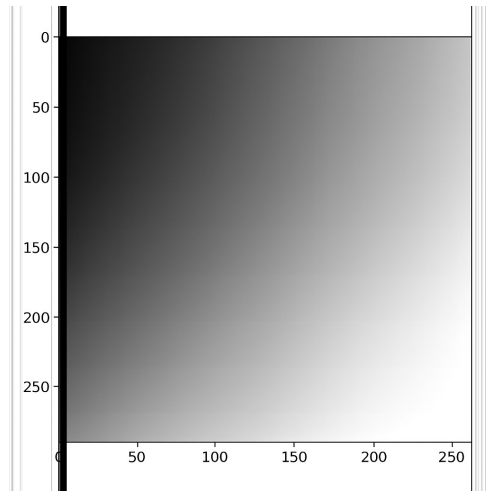


Fig. 4

Error Analysis The error was calculated using the Frobenius norm:

$$E_k = \|A - A_k\|_F = \sqrt{\sum_{i,j} (A_{ij} - A_{ij}^k)^2}$$

1) For einstein.pgm

k value	Forbenius Error
5	19753.34
20	19375.30
50	15897.67
100	11766.56

2) For globe.pgm

k value	Forbenius Error
5	153547.11
20	152915.28
50	147821.30
100	140419.38

3) For greyscale.pgm

k value	Forbenius Error
5	189591.66
20	187534.898
100	173560.61
200	156640.28

Discussion and Trade-off

The k controls how many singular values are retained during image reconstruction. A smaller k gives higher compression but results in loss of fine details and a blurrier image. A larger k preserves more detail and improves image quality, but requires more storage. Thus, k balances between quality and compression efficiency.

Run time and Accuracy

1) For einstein.pgm (run time = 0.003)

k value	Accuracy in percentage
10	85.10
50	95.96
100	99.24

2) For globe.pgm (runtime = 0.052)

k value	Accuracy
10	90.49
50	96.09
100	97.68

3) For greyscale.pgm (runtime = 0.12)

k value	Accuracy
10	96.29
50	99.39
100	99.70

Conclusion

The project successfully implemented image compression using the truncated SVD with the Jacobian method.